

Integrating COMA in the Deductive Verification of OCaml programs

Mário Pereira

version of July 15, 2025

Abstract

In this Master’s thesis, we propose to enhance the CAMELEER tool with a new pipeline for the deductive verification of OCAML programs. In particular, we aim to devise a translation schema from OCAML-annotated programs into COMA, a recently proposed intermediate verification language featuring explicit abstraction barriers. Leveraging on the features and strengths of COMA, we expect to use this approach to verify a relevant set of OCAML programs, while observing a significant reduction in the number of needed lines of specification.

1 Context

CAMELEER [6] is a tool for the automated deductive verification of OCAML programs. The typical usage of CAMELEER is as follows: first, the user provides formal specification for an OCAML implementation using the GOSPEL [1] specification language; then, CAMELEER triggers a translation step from OCAML annotated code into an equivalent program written in WHYML, the specification and verification language of the WHY3 framework [3]; finally, WHY3 generates Verification Conditions for the translated WHYML program, which imply the original OCAML program is correct with respect to the given GOSPEL specification.

To this point, CAMELEER has been used with success to formally verify the correctness of more than 1000 lines of OCAML code¹. Such implementations include purely functional data structures (*e.g.*, amortized queue), efficient algorithms (*e.g.*, union find), and functorial-based modules (*e.g.*, leftist heaps). CAMELEER has also been used as a vehicle to conduct research on deductive verification of OCAML programs and extensions to the GOSPEL language. These include a defunctionalization-based technique for the verification of control effects handlers [7] and a modular approach to reason about higher-order iterators [2].

¹<https://github.com/ocaml-gospel/cameleer/tree/master/examples>

2 Research Questions

In the scope of this thesis, we aim to implement an alternative workflow within the CAMELEER tool. Instead of relying exclusively on the OCAML to WHYML interface, we will build a pipeline that translates OCAML code to COMA [4], a recently proposed intermediate verification language.

In COMA, programs are written in the Continuation-Passing Style (CPS). This makes the language simple enough (only a few constructions are needed to capture standard control structures, such as loops or function calls), yet expressive enough to serve as a practical intermediate verification language for realistic scenarios. Specifications within COMA are mixed with executable code, while *abstraction barriers* are made explicit and allow one to separate the *interface* part of a function (verified at call site), from the *implementation* side (verified only once at definition site). COMA is already successfully used as an backend language for the verification of RUST and x86-64 programs. Hence, extending this ecosystem with a frontend for OCAML programs is, in our opinion, of utmost interest.

Leveraging on COMA explicit abstraction barriers, our goal is to explore the feasibility of this new verification workflow and compare it with the currently implemented WHYML-targeting one. In particular, we expect to achieve a considerable reduction in the number of lines of specification needed to complete the proof of realistic OCAML modules.

Our research question is two-fold:

- Can we build a translation pipeline from GOSPEL-specified OCAML code into the COMA intermediate verification language?
- Does such a pipeline allows one to achieve mostly-automated verification of OCAML modules, while reducing the number of needed lines of specification?

3 Work Plan

CPS-translation of OCAML programs. Following the nature of COMA, the first step in our pipeline is to build an automatic translation from OCAML programs into CPS equivalent implementations. We will build such a translation as an intermediate rewriting step directly within CAMELEER, as the tool already works with the untaped AST of the OCAML compiler.

Flexible barriers and GOSPEL specification. The second step of this work plan is the actual translation of (CPS-)OCAML code, together with GOSPEL annotations, into COMA. At this point, we will need to investigate how to include the explicit abstract flexible barriers into our GOSPEL-based

specification. The careful inclusion of the abstract barriers into the translation schema will be crucial to achieve the desired reduction in the number of lines of specification.

Case studies. The last step in this Master’s thesis is to exercise the devised verification pipeline, by building a comprehensive set of case studies. At this point, we will build new examples (*i.e.*, not proved before using CAMELEER), but will mostly and foremost consider examples from the CAMELEER and WHY3 galleries of verified programs. This will serve as a comparison test-bed: we will take the already existing specification of such examples and, using the newly developed OCAML to COMA pipeline, refactor it to fit the COMA-based approach. In particular, we will search for examples where the existing specification repeats most of the case-analysis structured done in the code, *e.g.*, the *Just Join* example by Filliâtre and Patault². This is where we expect to observe a significant reduction in the number of lines of specification, when comparing the WHYML-targeting translation to the one targeting COMA.

Advanced topics. If time allows, we can consider studying the following more advance topics:

- Dealing with OCAML programs featuring mutable side-effects. In the core work plan presented above, we are only considering purely functional OCAML implementations.
- Higher-order functions. One interesting application of COMA, given its CPS nature, is to infer the specification of simple higher-order functions [5]. This is, undoubtedly, an important topic when tackling the verification of OCAML programs, since higher-order functions are pervasive in idiomatic OCAML programs.

4 Research Environment

The candidate enrolled in this Master’s thesis will be a member of the GOSPEL project, a research project funded by the French National Research Agency, whose goal is to develop an ecosystem of tools and techniques to specify, test, and formally verify OCAML programs. NOVA LINCS, through the supervisor of this thesis, is a partner of the GOSPEL project.

The candidate will work in close collaboration with the core COMA team, namely Paul Patault, Andrei Paskevich, and Jean-Christophe Filliâtre. We have already agreed upon exchange visits, either at NOVA as well as at the LMF (*Laboratoire Méthodes Formelles – Univ. Paris-Saclay*).

²https://toccata.gitlabpages.inria.fr/toccata/gallery/just_join.en.html

References

- [1] Arthur Charguéraud, Jean-Christophe Filliâtre, Cláudio Lourenço, and Mário Pereira. GOSPEL - providing ocaml with a formal specification language. In Maurice H. ter Beek, Annabelle McIver, and José N. Oliveira, editors, *Formal Methods - The Next 30 Years - Third World Congress, FM 2019, Porto, Portugal, October 7-11, 2019, Proceedings*, volume 11800 of *Lecture Notes in Computer Science*, pages 484–501. Springer, 2019.
- [2] Ion Chirica and Mário Pereira. Unfolding iterators: Specification and verification of higher-order iterators, in ocaml. *CoRR*, abs/2506.20310, 2025.
- [3] Jean-Christophe Filliâtre and Andrei Paskevich. Why3 - where programs meet provers. In Matthias Felleisen and Philippa Gardner, editors, *Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings*, volume 7792 of *Lecture Notes in Computer Science*, pages 125–128. Springer, 2013.
- [4] Andrei Paskevich, Paul Patault, and Jean-Christophe Filliâtre. coma, an intermediate verification language with explicit abstraction barriers. In Viktor Vafeiadis, editor, *Programming Languages and Systems - 34th European Symposium on Programming, ESOP 2025, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2025, Hamilton, ON, Canada, May 3-8, 2025, Proceedings, Part II*, volume 15695 of *Lecture Notes in Computer Science*, pages 175–201. Springer, 2025.
- [5] Paul Patault, Arnaud Golfouse, and Xavier Denis. Remonter les barrières pour ouvrir une clôture. In *36es Journées Francophones des Langages Applicatifs (JFLA 2025)*, 2025.
- [6] Mário Pereira and António Ravara. Cameleer: A deductive verification tool for ocaml. In Alexandra Silva and K. Rustan M. Leino, editors, *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part II*, volume 12760 of *Lecture Notes in Computer Science*, pages 677–689. Springer, 2021.
- [7] Tiago Soares and Mário Pereira. A framework for the automated verification of algebraic effects and handlers (extended version). *CoRR*, abs/2302.01265, 2023.